

Advanced Graph Problems

Tree Traversals

```
#include

using namespace std;
const int maxN = 1e5+5;

int N, timer, A[maxN], B[maxN], C[maxN], idx[maxN];
int L[maxN], R[maxN];

int solve(int l1 = 0, int r1 = N-1, int l2 = 0, int r2 = N-1){
    if(l1 > r1 || l2 > r2) return 0;
    int root = A[l1];
    int m2 = idx[root];
    int len = m2-l2;
    L[root] = solve(l1+1, l1+len, l2, m2-1);
    R[root] = solve(l1+len+1, r1, m2+1, r2);
    return root;
}

void dfs(int u){
    if(L[u]) dfs(L[u]);
    if(R[u]) dfs(R[u]);
    C[timer++] = u;
}

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++) scanf("%d", &A[i]);
    for(int i = 0; i < N; i++){
        scanf("%d", &B[i]);
        idx[B[i]] = i;
    }

    int root = solve();
    dfs(root);
    for(int i = 0; i < N; i++)
        printf("%d%c", C[i], (" \n")[i==N-1]);
}
```

Course Schedule II

```
#include

using namespace std;
const int maxN = 2e5;

int N, M, a, b, idx, in[maxN], ans[maxN];
vector G[maxN];
priority_queue Q;

int main(){
```

```

scanf("%d %d", &N, &M);
for(int i = 0; i < M; i++){
    scanf("%d %d", &a, &b);
    G[b].push_back(a);
    in[a]++;
}

for(int i = 1; i <= N; i++)
    if(in[i] == 0)
        Q.push(i);

idx = N;
while(!Q.empty()){
    int u = Q.top(); Q.pop();
    ans[idx--] = u;
    for(int v : G[u]){
        in[v]--;
        if(in[v] == 0)
            Q.push(v);
    }
}

for(int i = 1; i <= N; i++)
    printf("%d%c", ans[i], (" \n")[i==N]);
}

```

Acyclic Graph Edges

```

#include

using namespace std;
int N, M, a, b;

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        printf("%d %d\n", min(a, b), max(a, b));
    }
}

```

Strongly Connected Edges

```

#include

using namespace std;
typedef pair pii;
const int maxN = 1e5+1;
const int maxM = 2e5+1;

int N, M, timer, tin[maxN], low[maxN];
bool bridge_exists, vis[maxN], used[maxM];

```

```

vector G[maxN];
pii edges[maxM];

void dfs(int u = 1, int p = 0){
    vis[u] = true;
    tin[u] = low[u] = ++timer;
    for(pii P : G[u]){
        int v = P.first;
        int id = P.second;
        if(used[id]) continue;

        used[id] = true;
        edges[id] = {u, v};

        if(vis[v]){
            low[u] = min(low[u], tin[v]);
        } else {
            dfs(v, u);
            low[u] = min(low[u], low[v]);
            if(low[v] > tin[u])
                bridge_exists = true;
        }
    }
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back({b, i});
        G[b].push_back({a, i});
    }

    dfs();
    if(timer == N && !bridge_exists){
        for(int i = 0; i < M; i++)
            printf("%d %d\n", edges[i].first, edges[i].second);
    } else printf("IMPOSSIBLE\n");
}

```

Even Outdegree Edges

```

#include

using namespace std;
typedef pair pii;
const int maxN = 1e5+5, maxM = 2e5+5;

int N, M, timer, in[maxN];
bool even[maxN];
vector G[maxN];
pii ans[maxM];

void dfs(int u, int p = -1){
    in[u] = ++timer;
    for(pii e : G[u]){
        int v = e.first, id = e.second;
        if(v != p){
            if(!in[v]){
                dfs(v, u);
                if(even[v]){

```

```

        ans[id] = {u, v};
        even[u] ^= true;
    } else {
        ans[id] = {v, u};
        even[v] ^= true;
    }
} else if(in[v] < in[u]){
    even[u] ^= true;
    ans[id] = {u, v};
}
}
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back({b, i});
        G[b].push_back({a, i});
    }

    fill(even+1, even+N+1, true);
    for(int i = 1; i <= N; i++)
        if(!in[i])
            dfs(i);

    bool good = true;
    for(int i = 1; i <= N; i++)
        good &= even[i];

    if(good){
        for(int i = 0; i < M; i++)
            printf("%d %d\n", ans[i].first, ans[i].second);
    } else printf("IMPOSSIBLE\n");
}

```

Graph Girth

```

#include

using namespace std;
const int maxN = 2501;

bool vis[maxN];
int N, M, best, p[maxN], dist[maxN];
vector G[maxN];

void reset(){
    fill(vis+1, vis+N+1, false);
    fill(dist+1, dist+N+1, 0);
    fill(p+1, p+N+1, 0);
}

void bfs(int start){
    queue Q;
    Q.push(start);
    vis[start] = true;
    while(!Q.empty()){
        int u = Q.front(); Q.pop();
        for(int v : G[u]){

```

```

        if(!vis[v]){
            dist[v] = dist[u]+1;
            vis[v] = true;
            p[v] = u;
            Q.push(v);
        } else if(v != u[p]){
            best = min(best, dist[u]+dist[v]+1);
        }
    }
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    best = N+1;
    for(int start = 1; start <= N; start++){
        reset();
        bfs(start);
    }

    printf("%d\n", best == N+1 ? -1 : best);
}

```

Transfer Speeds Sum

```

#include

using namespace std;
typedef long long ll;
typedef array edge;
const int maxN = 2e5+1;

int N, ds[maxN];
ll ans;
vector edges;

int find(int u){
    if(ds[u] < 0) return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v) return false;
    if(ds[u] < ds[v]) swap(u, v);
    ds[v] += ds[u];
    ds[u] = v;
    return true;
}

int main(){
    scanf("%d", &N);
    for(int i = 0, a, b, x; i < N-1; i++){
        scanf("%d %d %d", &a, &b, &x);
    }
}

```

```

        edges.push_back({a, b, x});
    }

    sort(edges.begin(), edges.end(), [](const edge& e1, const edge& e2){
        return e1[2] > e2[2];
    });

    fill(ds, ds+N, -1);
    for(const edge& e: edges){
        int a = find(e[0]), b = find(e[1]);
        int sza = -ds[a], szb = -ds[b];

        ll x = e[2];
        ans += x * sza * szb;
        merge(a, b);
    }
    printf("%lld\n", ans);
}

```

Network Breakdown

```

#include

using namespace std;
const int maxN = 1e5+1;
const int maxM = 2e5;
typedef pair pii;

int N, M, K, a, b, cnt, ds[maxN], ans[maxM];
pii edges[maxM], queries[maxM];
set S;

int find(int u){
    if(ds[u] < 0) return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v) return false;
    if(ds[u] < ds[v]) swap(u, v);
    ds[v] += ds[u];
    ds[u] = v;
    return true;
}

int main(){
    scanf("%d %d %d", &N, &M, &K);
    fill(ds+1, ds+N+1, -1);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        if(b > a) swap(a, b);
        edges[i] = {a, b};
    }
    for(int i = 0; i < K; i++){
        scanf("%d %d", &a, &b);
        if(b > a) swap(a, b);
        queries[i] = {a, b};
        S.insert({a, b});
    }
}

```

```

cnt = N;
for(int i = 0; i < M; i++)
    if(S.find(edges[i]) == S.end())
        if(merge(edges[i].first, edges[i].second))
            cnt--;

for(int i = K-1; i >= 0; i--){
    ans[i] = cnt;
    if(merge(queries[i].first, queries[i].second))
        cnt--;
}

for(int i = 0; i < K; i++)
    printf("%d%c", ans[i], (" \n")[i==K-1]);
}

```

Tree Isomorphism I

```

#include

using namespace std;
const int maxN = 1e5+1;

int N, treeID, name[2][maxN];
vector G[2][maxN];
map<int> mp;

void reset(){
    for(int t = 0; t < 2; t++)
        for(int i = 1; i <= N; i++)
            G[t][i].clear();
}

void dfs(int t, int u, int p){
    vector childNames;
    for(int v : G[t][u]){
        if(v != p){
            dfs(t, v, u);
            childNames.push_back(name[t][v]);
        }
    }
    sort(childNames.begin(), childNames.end());
    if(!mp[childNames]) mp[childNames] = ++treeID;
    name[t][u] = mp[childNames];
}

void solve_case(){
    scanf("%d", &N);
    reset();
    for(int t = 0; t < 2; t++){
        for(int i = 0, a, b; i < N-1; i++){
            scanf("%d %d", &a, &b);
            G[t][a].push_back(b);
            G[t][b].push_back(a);
        }
        dfs(t, 1, -1);
    }
    printf("%s\n", name[0][1] == name[1][1] ? "YES" : "NO");
}

```

```
int main(){
    int T;
    scanf("%d", &T);
    for(int t = 0; t < T; t++){
        solve_case();
    }
}
```

Tree Isomorphism II

```
#include

using namespace std;
const int maxN = 1e5+1;

int N, treeID, sz[2][maxN], name[2][maxN];
vector centroids[2], G[2][maxN];
map<int> mp;

void reset(){
    mp.clear();
    treeID = 0;
    for(int t = 0; t < 2; t++){
        centroids[t].clear();
        for(int i = 1; i <= N; i++){
            sz[t][i] = name[t][i] = 0;
            G[t][i].clear();
        }
    }
}

void dfs1(int t, int u, int p){
    sz[t][u] = 1;
    bool is_centroid = true;
    for(int v : G[t][u]){
        if(v != p){
            dfs1(t, v, u);
            sz[t][u] += sz[t][v];
            if(sz[t][v] > N/2) is_centroid = false;
        }
    }
    if(N-sz[t][u] > N/2) is_centroid = false;
    if(is_centroid) centroids[t].push_back(u);
}

void dfs2(int t, int u, int p){
    vector childNames;
    for(int v : G[t][u]){
        if(v != p){
            dfs2(t, v, u);
            childNames.push_back(name[t][v]);
        }
    }
    sort(childNames.begin(), childNames.end());
    if(!mp[childNames]) mp[childNames] = ++treeID;
    name[t][u] = mp[childNames];
}

void solve_case(){
    scanf("%d", &N);
    reset();
```

```

for(int t = 0; t < 2; t++){
    for(int i = 0, a, b; i < N-1; i++){
        scanf("%d %d", &a, &b);
        G[t][a].push_back(b);
        G[t][b].push_back(a);
    }
    dfs1(t, 1, -1);
}

for(int root1 : centroids[0]){
    for(int root2 : centroids[1]){
        dfs2(0, root1, -1);
        dfs2(1, root2, -1);
        if(name[0][root1] == name[1][root2]){
            printf("YES\n");
            return;
        }
    }
}
printf("NO\n");
}

int main(){
    int T;
    scanf("%d", &T);
    for(int t = 0; t < T; t++){
        solve_case();
    }
}

```

Flight Route Requests

```

#include

using namespace std;
const int maxN = 1e5+1;

int N, M, ds[maxN], vis[maxN];
vector G[maxN], CC[maxN];

int find(int u){
    if(ds[u] < 0) return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v) return false;
    if(ds[u] < ds[v]) swap(u, v);
    ds[v] += ds[u];
    ds[u] = v;
    return true;
}

bool dfs(int u){
    vis[u] = -1;
    bool hascycle = false;
    for(int v : G[u]){
        if(vis[v] == -1) return true;
    }
}

```

```

        else if(vis[v] == 0)    hascycle |= dfs(v);
    }
    vis[u] = 1;
    return hascycle;
}

int main(){
    scanf("%d %d", &N, &M);

    fill(ds+1, ds+N+1, -1);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        merge(a, b);
    }

    int K = 0;
    unordered_map getID;
    for(int u = 1; u <= N; u++){
        int rep = find(u);
        if(!getID[rep])
            getID[rep] = ++K;
        CC[getID[rep]].push_back(u);
    }

    int ans = 0;
    for(int k = 1; k <= K; k++){
        int sz = (int) CC[k].size();
        bool hascycle = false;
        for(int u : CC[k])
            if(!hascycle && vis[u] == 0)
                hascycle |= dfs(u);
        ans += (hascycle ? sz : sz-1);
    }
    printf("%d\n", ans);
}

```

Critical Cities

```

#include

using namespace std;
const int maxN = 1e5+1;

int N, M, p[maxN], sdom[maxN], idom[maxN], label[maxN];
int timer, tin[maxN], inv[maxN], dsu[maxN];
vector ans, G[maxN], GR[maxN], DT[maxN], bucket[maxN];

void dfs(int u = 1){
    tin[u] = ++timer;
    inv[timer] = u;
    label[timer] = sdom[timer] = dsu[timer] = timer;
    for(int v : G[u]){
        if(!tin[v]){
            dfs(v);
            p[tin[v]] = tin[u];
        }
        GR[tin[v]].push_back(tin[u]);
    }
}

```

```

int find(int u, bool x = false){
    if(u == dsu[u]) return x ? -1 : u;
    int v = find(dsu[u], true);
    if(v < 0) return u;

    if(sdom[label[dsu[u]]] < sdom[label[u]])
        label[u] = label[dsu[u]];

    dsu[u] = v;
    return x ? v : label[u];
}

void build_dominator_tree(){
    dfs();

    for(int u = N; u > 0; u--){
        for(int v : GR[u])
            sdom[u] = min(sdom[u], sdom[find(v)]);
        if(u > 1) bucket[sdom[u]].push_back(u);
        for(int v : bucket[u])
            idom[v] = (sdom[find(v)] == sdom[v] ? sdom[v] : find(v));
        if(u > 1) dsu[u] = p[u];
    }

    for(int u = 2; u <= N; u++){
        if(idom[u] != sdom[u])
            idom[u] = idom[idom[u]];
        DT[inv[u]].push_back(inv[idom[u]]);
        DT[inv[idom[u]]].push_back(inv[u]);
    }
}

bool dfs_dominator_tree(int u = 1, int p = -1){
    bool good = (u == N);
    for(int v : DT[u])
        if(v != p)
            good |= dfs_dominator_tree(v, u);
    if(good) ans.push_back(u);
    return good;
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
    }

    build_dominator_tree();
    dfs_dominator_tree();

    sort(ans.begin(), ans.end());
    int K = (int) ans.size();
    printf("%d\n", K);
    for(int i = 0; i < K; i++)
        printf("%d%c", ans[i], (" \n")[i==K-1]);
}

```

Visiting Cities

```

#include

using namespace std;
typedef long long ll;
typedef pair edge;
typedef pair node;
const int maxN = 1e5+1;
const ll INF = 0x3f3f3f3f3f3f3f3f;

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
uniform_int_distribution distrib((ll) 1e9, (ll) 2e9);
const ll MOD1 = distrib(rng);
const ll MOD2 = distrib(rng);

int N, M;
ll dist[2][maxN], ways[2][2][maxN];
vector G[2][maxN];
vector ans;

void init(){
    for(int t = 0; t < 2; t++){
        for(int i = 1; i <= N; i++){
            dist[t][i] = INF;
        }
    }

    void dijkstra(int type, int source){
        priority_queue, greater> Q;
        dist[type][source] = 0;
        ways[type][0][source] = 1;
        ways[type][1][source] = 1;
        Q.push({0, source});
        while(!Q.empty()){
            int u = Q.top().second;
            ll d = Q.top().first;
            Q.pop();

            if(d > dist[type][u]) continue;
            for(edge e : G[type][u]){
                int v = e.first;
                ll w = e.second;
                if(dist[type][v] > d+w){
                    ways[type][0][v] = ways[type][0][u];
                    ways[type][1][v] = ways[type][1][u];
                    dist[type][v] = d+w;
                    Q.push({d+w, v});
                } else if(dist[type][v] == d+w){
                    ways[type][0][v] += ways[type][0][u];
                    ways[type][0][v] %= MOD1;
                    ways[type][1][v] += ways[type][1][u];
                    ways[type][1][v] %= MOD2;
                }
            }
        }
    }
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b, c; i < M; i++){
        scanf("%d %d %d", &a, &b, &c);
        G[0][a].push_back({b, c});
        G[1][b].push_back({a, c});
    }
    init();
    dijkstra(0, 1);
    dijkstra(1, N);

    ll tot1 = ways[0][0][N], tot2 = ways[0][1][N];
    for(int u = 1; u <= N; u++){
        ll ways1 = ways[0][0][u] * ways[1][0][u] % MOD1;

```

```

ll ways2 = ways[0][1][u] * ways[1][1][u] % MOD2;
if(ways1 == tot1 && ways2 == tot2 && dist[0][u] + dist[1][u] == dist[0][N])
    ans.push_back(u);
}

int K = (int) ans.size();
printf("%d\n", K);
for(int i = 0; i < K; i++)
    printf("%d%c", ans[i], (" \n")[i==K-1]);
}

```

Network Renovation

```

#include

using namespace std;
const int maxN = 1e5+1;

int N, K;
vector leaves, G[maxN];

void dfs(int u = 1, int p = -1){
    if((int) G[u].size() == 1) leaves.push_back(u);
    for(int v : G[u])
        if(v != p)
            dfs(v, u);
}

int main(){
    scanf("%d", &N);
    for(int i = 0, a, b; i < N-1; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    dfs();

    int K = (int) leaves.size();
    printf("%d\n", (K+1)/2);
    for(int i = 0; i < (K+1)/2; i++)
        printf("%d %d\n", leaves[i], leaves[i+K/2]);
}

```

Creating Offices

```

#include

using namespace std;
const int maxN = 2e5+1, logN = 20;
const int INF = 0x3f3f3f3f;

```

```

int N, D, ct[maxN], sz[maxN], best[maxN];
int timer, p[maxN][logN], d[maxN], in[maxN], out[maxN];
bool vis[maxN];
vector G[maxN];

void dfs(int u = 1, int par = 1){
    in[u] = ++timer;
    d[u] = d[par]+1;
    p[u][0] = par;
    for(int i = 1; i < logN; i++)
        p[u][i] = p[p[u][i-1]][i-1];
    for(int v : G[u])
        if(v != par)
            dfs(v, u);
    out[u] = ++timer;
}

bool ancestor(int u, int v){
    return in[u] <= in[v] && out[u] >= out[v];
}

int lca(int u, int v){
    if(ancestor(u, v)) return u;
    if(ancestor(v, u)) return v;
    for(int i = logN-1; i >= 0; i--)
        if(!ancestor(p[u][i], v))
            u = p[u][i];
    return p[u][0];
}

int dist(int u, int v){
    return d[u] + d[v] - 2*d[lca(u, v)];
}

int find_size(int u, int p = -1){
    if(vis[u]) return 0;

    sz[u] = 1;
    for(int v : G[u])
        if(v != p)
            sz[u] += find_size(v, u);

    return sz[u];
}

int find_centroid(int u, int p, int n){
    for(int v : G[u])
        if(v != p)
            if(!vis[v] && sz[v] > n/2)
                return find_centroid(v, u, n);

    return u;
}

void build_centroid_tree(int u = 1, int p = -1){
    find_size(u);

    int c = find_centroid(u, -1, sz[u]);
    vis[c] = true;
    ct[c] = p;

    for(int v : G[c])
        if(!vis[v])
            build_centroid_tree(v, c);
}

void update(int u){
    best[u] = 0;
    int v = u;
    while(ct[v] != -1){

```

```

        v = ct[v];
        best[v] = min(best[v], dist(u, v));
    }
}

int query(int u){
    int ans = best[u];
    int v = u;
    while(ct[v] != -1){
        v = ct[v];
        ans = min(ans, best[v] + dist(u, v));
    }
    return ans;
}

int main(){
    scanf("%d %d", &N, &D);
    for(int i = 0, a, b; i < N-1; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    dfs();
    build_centroid_tree();
    fill(best+1, best+N+1, INF);

    vector order(N);
    for(int i = 0; i < N; i++)
        order[i] = i+1;
    sort(order.begin(), order.end(), [](int a, int b){
        return d[a] == d[b] ? a < b : d[a] > d[b];
    });

    vector ans;
    for(int u : order){
        int dist_to_office = query(u);
        if(dist_to_office >= D){
            ans.push_back(u);
            update(u);
        }
    }

    sort(ans.begin(), ans.end());
    int K = (int) ans.size();
    printf("%d\n", K);
    for(int i = 0; i < K; i++)
        printf("%d%c", ans[i], (" \n")[i==K-1]);
}

```